

Sensor-Based IoT Simulation

Temperature-Controlled LED using Arduino Uno and TMP36 in Tinkercad Circuits

Internship Task 2 | Prepared by: Praharsha | October 2026

1. Objective

To simulate a simple IoT sensing-and-actuation system in Tinkercad Circuits, without any physical hardware. A TMP36 temperature sensor feeds an Arduino Uno, which switches an LED on when the temperature rises above 30°C and off when it falls back. The same sense, process, act pattern is the core of larger IoT systems such as smart fans, cold-chain alerts and greenhouse controllers.

2. Components Used

Component	Qty	Purpose
Arduino Uno R3	1	Microcontroller: reads the sensor, decides, drives the LED
TMP36 temperature sensor	1	Outputs an analog voltage proportional to temperature
LED (red)	1	Output indicator (actuator)
Resistor, 220 ohm	1	Limits LED current
Breadboard (optional)	1	Convenient place to join the wires

3. Circuit Connections

From	To
TMP36 pin 1 (+Vs)	Arduino 5V
TMP36 pin 2 (Vout)	Arduino A0
TMP36 pin 3 (GND)	Arduino GND
LED anode (longer leg)	220 ohm resistor, then Arduino D8
LED cathode (shorter leg)	Arduino GND

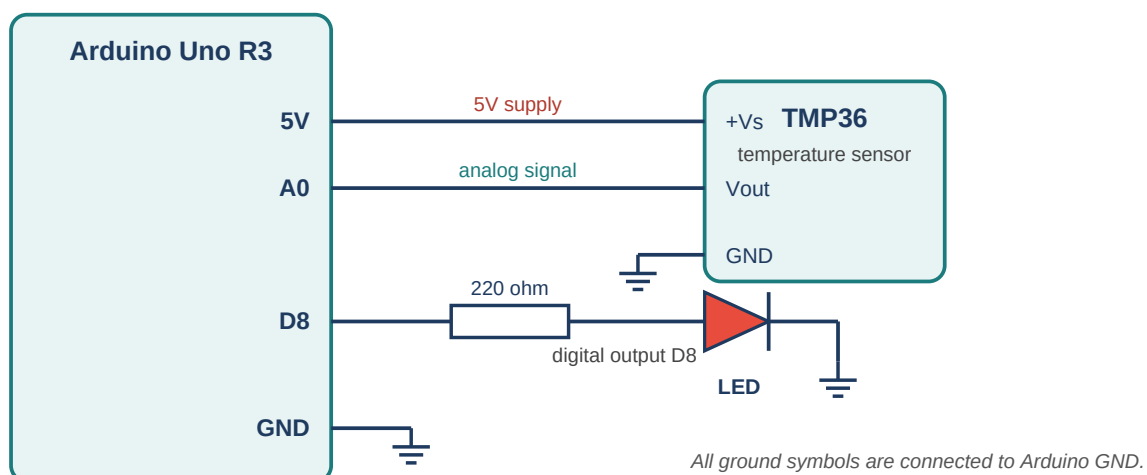


Figure 1. Circuit diagram of the temperature-controlled LED.

4. Steps to Build and Run the Simulation

1. Open tinkercad.com, sign in, choose **Circuits** and click **Create new Circuit**.
2. Drag an **Arduino Uno R3**, a **TMP36 temperature sensor**, an **LED** and a **Resistor** (set to 220 ohm) onto the workspace.
3. Wire the parts exactly as in Section 3 (with the TMP36 flat side facing you, the pins from left to right are +Vs, Vout and GND).
4. Click **Code**, switch the editor to **Text**, delete the sample code and paste the program from Section 5.
5. Click **Start Simulation**, open the **Serial Monitor**, then click the TMP36 and drag its temperature slider above and below 30°C. The LED should switch accordingly.

5. Arduino Code

```
// Temperature-controlled LED (TMP36 + Arduino Uno) - Tinkercad simulation
const int sensorPin = A0;           // TMP36 output connected to analog pin A0
const int ledPin = 8;               // LED (through 220 ohm resistor) on digital pin 8
const float threshold = 30.0;       // temperature limit in degrees Celsius

void setup() {
  pinMode(ledPin, OUTPUT);          // LED pin is an output
  Serial.begin(9600);               // start serial monitor at 9600 baud
}

void loop() {
  int reading = analogRead(sensorPin); // ADC value, 0 to 1023
  float voltage = reading * 5.0 / 1024.0; // convert ADC value to volts
  float temperatureC = (voltage - 0.5) * 100.0; // TMP36: 10 mV/degC, 0.5 V offset

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.print(" C | LED: ");

  if (temperatureC > threshold) {
    digitalWrite(ledPin, HIGH);      // too hot -> LED ON
    Serial.println("ON");
  } else {
    digitalWrite(ledPin, LOW);       // normal -> LED OFF
    Serial.println("OFF");
  }
  delay(500);                        // read twice per second
}
```

Listing 1. Complete sketch.

6. Code Explanation

Part of the code	What it does
sensorPin, ledPin, threshold	Named constants for the sensor pin (A0), the LED pin (D8) and the 30°C limit, so they are easy to change in one place.
pinMode(ledPin, OUTPUT)	Configures D8 as an output so the Arduino can drive the LED.
Serial.begin(9600)	Opens the serial link so readings appear in the Serial Monitor.
analogRead(sensorPin)	The 10-bit ADC converts the 0 to 5 V sensor voltage into a number from 0 to 1023.
reading * 5.0 / 1024.0	Converts that number back to volts.
(voltage - 0.5) * 100.0	The TMP36 gives 10 mV per °C with a 0.5 V offset at 0°C, so subtracting 0.5 V and multiplying by 100 gives degrees Celsius.

Part of the code	What it does
if (temperatureC > threshold)	The decision step: above 30°C the LED is set HIGH (on), otherwise LOW (off).
delay(500)	Waits half a second between readings.

7. How This Works as an IoT System

Sense: the TMP36 measures the environment. **Process:** the Arduino converts and compares the reading with a threshold. **Act:** the LED output switches on or off. **Report:** the Serial Monitor shows the live value, standing in for the cloud dashboard of a real deployment. Replacing the Arduino with an ESP32 or ESP8266 would let the same logic publish its readings over Wi-Fi using MQTT.

8. Expected Behaviour

The table lists what the program should do for different slider settings. Compare it with your own results and screenshots.

Sensor temperature set to	Serial Monitor line (approx.)	LED
25°C	Temperature: 25.00 C LED: OFF	OFF
29°C	Temperature: 29.00 C LED: OFF	OFF
31°C	Temperature: 31.00 C LED: ON	ON
40°C	Temperature: 40.00 C LED: ON	ON
Back to 25°C	Temperature: 25.00 C LED: OFF	OFF

Table 1. Expected results. Printed values may differ by a degree or so because the ADC has limited resolution.

9. Simulation Screenshots

Capture these four screenshots from your own Tinkercad run and paste them over the boxes below.

Figure 2: Circuit built in Tinkercad

Full workspace showing Arduino, TMP36, resistor, LED and all wires

[paste your Tinkercad screenshot here]

Figure 3: LED OFF

Temperature slider at 25 C, LED dark, Serial Monitor showing OFF

[paste your Tinkercad screenshot here]

Figure 4: LED ON

Temperature slider at 40 C, LED lit, Serial Monitor showing ON

[paste your Tinkercad screenshot here]

Figure 5: Code and Serial Monitor

Code editor open with the Serial Monitor output visible

[paste your Tinkercad screenshot here]

10. Conclusion and Possible Improvements

The simulation shows a complete sensor-to-actuator loop that behaves correctly without hardware. Possible upgrades are a second LED or buzzer for a higher alert level, a small hysteresis band (for example on at 30°C and off at 28°C) to stop flickering near the threshold, and a Wi-Fi microcontroller to send readings to a cloud dashboard.